

Software Requirements Specification (SRS)

Heal6: Clinical-Grade Diabetic Foot Analysis Platform

Version: 1.0

Date: August 29, 2026

Target: Smart India Hackathon

1. Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for **Heal6**, an enterprise-grade AI telemetry and triage platform for Diabetic Foot Ulcers (DFU). It details the functional and non-functional requirements for the Patient Edge Application, the ML/FastAPI Backend, and the Physician Command Center.

1.2 Scope

Heal6 is a full-stack clinical intelligence suite designed to accelerate DFU triage, automate surface area metrology, and predict infection risks. It acts as an early-intervention system to reduce major amputation rates. The system consists of:

1. **Patient Edge Client:** A secure web/mobile interface for patients to upload wound images using 25mm ArUco fiducial markers and perform guided clinical self-assessments (e.g., capillary/twig tests).
2. **ML Telemetry Backend:** A Python/FastAPI server utilizing state-of-the-art (SOTA) computer vision (ConvNeXt-V2 and UNet++) for optical homography, tissue segmentation, and calculating the IWGDF SINBAD score.
3. **Clinician Workstation:** A React-based, dark-mode dashboard for podiatrists to review the Master Triage Queue, validate AI telemetry via Human-in-the-Loop (HITL) overrides, and issue digital treatment directives.

1.3 Definitions, Acronyms, and Abbreviations

- **DFU:** Diabetic Foot Ulcer
- **SINBAD:** Site, Ischemia, Neuropathy, Bacterial Infection, Area, Depth (Clinical scoring matrix).
- **IWGDF:** International Working Group on the Diabetic Foot.
- **ArUco:** Synthetic square fiducial marker for camera pose estimation.
- **CV:** Computer Vision.

- **HITL:** Human-in-the-Loop.
- **WSOD:** White Screen of Death (Frontend failure state).

2. Overall Description

2.1 Product Perspective

Heal6 operates as a distributed web application. The frontend clients (Patient and Doctor) are built using React (Vite + TailwindCSS + Framer Motion) and communicate via REST API with a centralized FastAPI server. The backend hosts embedded PyTorch deep learning models to process image and metadata payloads.

2.2 User Characteristics

- **Patients:** Users with varying degrees of technical literacy. Require highly guided, minimalist, and accessible UI.
- **Physicians/Specialists:** Highly trained medical professionals requiring data-dense, dark-mode interfaces to reduce eye strain, capable of complex data interpretation and overriding AI decisions.

2.3 Operating Environment

- **Frontend:** Modern web browsers (Chrome, Safari, Edge) on mobile devices, tablets, and desktop workstations.
- **Backend:** Linux/Ubuntu-based cloud environment with CUDA-enabled GPU acceleration for PyTorch model inference.

3. Functional Requirements

3.1 Patient Intake & Acquisition Module

- **REQ-1.1:** The system shall allow patients to create/login to a profile capturing Age, Gender, and Diabetes History.
- **REQ-1.2:** The system shall capture clinical images, requiring the presence of an ArUco fiducial marker for scale calibration.
- **REQ-1.3:** The system shall present an interactive Clinical Self-Assessment for:
 - Site Location (Hindfoot/Midfoot)
 - Capillary Refill Test (Ischemia)
 - Touch/Twig Test (Neuropathy)
 - Wound Depth
- **REQ-1.4:** The system shall package the image and boolean assessment data into a unified JSON/Multipart payload and POST it to the backend.

3.2 ML Backend & Telemetry Pipeline

- **REQ-2.1:** The backend shall utilize OpenCV to detect ArUco Marker #42 (25mm) to establish a pixel-to-cm ratio (e.g., 42px/cm) and perform geometric orthorectification.

- **REQ-2.2:** The backend shall process the image through a ConvNeXt-V2 Large model to output a binary Abnormality/Infection probability percentage.
- **REQ-2.3:** The backend shall process the image through an Attention UNet++ (EfficientNet-B4 backbone) to generate a 4-class segmentation mask (Granulation, Slough, Necrotic, Background).
- **REQ-2.4:** The backend shall calculate the true wound surface area in cm².
- **REQ-2.5:** The backend shall compute a dynamic SINBAD score (0-6) by synthesizing patient inputs and AI outputs.
- **REQ-2.6:** The backend shall encode the predicted UNet++ segmentation mask into a Base64 PNG and return it in the API response.

3.3 Master Triage Queue (Doctor Portal)

- **REQ-3.1:** The workstation shall securely poll the backend to retrieve active patient cases.
- **REQ-3.2:** The system shall sort the Master Triage Queue dynamically in descending order based on the calculated SINBAD Severity Score.
- **REQ-3.3:** The system shall visually flag patients with a SINBAD score ≥ 4 as "Urgent" or "Critical".
- **REQ-3.4:** The system shall provide search and filter functionalities (e.g., Filter by "Granulating/Healing" vs "Critical").

3.4 Patient Command Center (Doctor Portal)

- **REQ-4.1:** The system shall render a dual-layer visual engine displaying the raw patient clinical scan overlaid with the Base64 PyTorch segmentation mask.
- **REQ-4.2:** The system shall provide an opacity slider for the doctor to adjust the visibility of the AI boundary mask.
- **REQ-4.3:** The system shall feature a "Human Validation" panel allowing the physician to override the AI/Patient inputs (e.g., manually confirming Pedal Pulses or Probe-to-Bone depth).
- **REQ-4.4:** The system shall dynamically update a 6-Axis SINBAD Radar chart based on the accepted clinical parameters.
- **REQ-4.5:** The system shall project a 12-week mathematical healing trajectory comparing the Heal6 Protocol vs. Standard Care.
- **REQ-4.6:** The system shall allow the doctor to digitally sign-off on the report, triggering the patient's removal from the active triage queue.

4. External Interface Requirements

4.1 User Interfaces

- **Patient App:** Light mode, card-based, mobile-first design leveraging Inter font and high-contrast Teal primary actions.
- **Doctor Web:** Dark-glass aesthetic (Glassmorphism), high-density data tables, and dynamic SVG graphics to mimic an industrial HIS (Health Information System) terminal.

4.2 Software Interfaces

- **FastAPI REST API:** Facilitates all communication between the React frontends and PyTorch models.
- **Export Formats:** The system shall generate downloadable, print-optimized .pdf formal clinical reports and .csv registry analytics exports.

5. Non-Functional Requirements

5.1 Performance Requirements

- **Pre-Warming:** The FastAPI lifespan context manager shall load ML models into memory at server boot to reduce inference latency from ~4.0s to < 350ms.
- **Zero-Crash Fallback:** The frontend React applications shall employ null-safe rendering and automatically hydrate the UI with high-fidelity local fallback data if the backend API is unreachable (preventing WSOD).

5.2 Security & Compliance

- **Data Ephemerality:** Raw Patient Identifiable Information (PII) and imagery shall not be permanently retained on external clouds without encryption.
- **Simulated Standards:** System architecture is aligned to eventually support HIPAA compliance and HL7/FHIR message routing for inter-hospital referrals.

5.3 Reliability & Availability

- The system shall provide real-time connection status polling to inform the clinician if they are viewing "Live Synced" data or "Local Telemetry".